DSA

Practical No. 5

Infix to postfix

Name: Arjun Mankuskar

PRN:202401110038

Branch: CSE (AIML)

Batch: A2

```
#include <iostream>
#include <cctype>   // for isdigit(), isalpha()
#include <cstring>  // for strcat()

#define SIZE 100
using namespace std;

char stackArr[SIZE];
int top = -1;

// Push operation
void push(char item) {
   if (top >= SIZE - 1) {
      cout << "\nStack Overflow.\n";
   } else {
      stackArr[++top] = item;
   }
}

// Pop operation
char pop() {
   if (top < 0) {
      cout << "Stack Underflow: Invalid infix expression.\n";
      exit(1);
```

```
    } else {
        return stackArr[top--];
    }
}

// Check if symbol is an operator
bool isOperator(char symbol) {
    return (symbol == '^' || symbol == '*' || symbol == '/' || symbol == '+' || symbol == '-');
}

// Return operator precedence
int precedence(char symbol) {
    if (symbol == '^')
        return 3;
    else if (symbol == '*' || symbol == '/')
        return 2;
    else if (symbol == '+' || symbol == '-')
        return 1;
    else
        return 0;
}

// Convert infix to postfix
```

```
void infixToPostfix(char infix[], char postfix[]) {
    int i = 0, j = 0;
    char item, x;

    push('(');
    strcat(infix, ")");

    item = infix[i];

    while (item != '\0') {
        if (item == '(') {
            push(item);
        }
        else if (isalnum(item)) {  // operand
            postfix[j++] = item;
        }
        else if (isOperator(item)) {
            x = pop();
            while (isOperator(x) && precedence(x) >= precedence(item)) {
                postfix[j++] = x;
                x = pop();
            }
            push(x);
            push(item);
```

```
        }
        else if (item == ')') {
            x = pop();
            while (x != '(') {
                postfix[j++] = x;
                x = pop();
            }
        }
        else {
            cout << "\nInvalid character in expression: " << item << endl;
            exit(1);
        }

        item = infix[++i];
    }

    if (top > 0) {
        cout << "\nInvalid infix expression (unbalanced parentheses).\n";
        exit(1);
    }

    postfix[j] = '\0';
}
```

```
// Main function
int main() {
    char infix[SIZE], postfix[SIZE];

    cout << "Enter Infix Expression: ";
    cin.getline(infix, SIZE);

    infixToPostfix(infix, postfix);

    cout << "Postfix Expression: " << postfix << endl;

    return 0;
}

/*
OUTPUT:-

Enter Infix Expression: a*(b+c)-d
Postfix Expression: abc+*d-
*/
```